

Redis Atomic Operations: Deep Dive into SET Options, Race Conditions, and Lua Scripting

Part 1: Understanding Redis SET Options

Option Trade-offs Matrix

Option	Behavior	TTL Reset?	When to Use
EX	Set with expiry in seconds	YES	Simple cache, short-lived data
PX	Set with expiry in milliseconds	YES	Sub-second precision needed
KEEPTTL	Preserve existing TTL	NO	Update value, keep expiry
NX	Only if key doesn't exist	N/A (fails if exists)	Create-only operations
XX	Only if key exists	N/A (fails if not exists)	Update-only operations
GET	Return old value atomically	YES (resets TTL)	Swap operations

Part 2: The Problem (Your Use Case)

Goal:

- First access: Create key with 30-second TTL
- Later access: Update value, preserve existing TTL
- **Must be atomic** to prevent race conditions

Why This Matters: In a rate limiter, a race condition could mean:

- Multiple requests see the same count
- TTL expires mid-request
- Double-counting requests
- Bypass of rate limit

Part 3: The Naive Approach (UNSAFE ❌)

Implementation:

```
// NAIVE: DO NOT USE IN PRODUCTION
async function naiveSetWithConditionalTTL(key, value) {
  // Step 1: Check if key exists
  const exists = await redis.exists(key); // Redis command 1

  // ⚠ RACE CONDITION WINDOW HERE ⚠
  // Between EXISTS and SET, another client could delete/update the key
```

```
// Step 2: Set based on existence
if (exists) {
  // Key exists, preserve TTL
  await redis.set(key, value, { KEeptTL: true }); // Redis command 2
} else {
  // Key doesn't exist, set with 30s TTL
  await redis.set(key, value, { EX: 30 }); // Redis command 2
}
}
```

Why It Fails:

Timeline of Race Condition:		
Client A Timeline	Client B Timeline	Redis State
		key: "foo" (TTL: 10s)
10s)		
EXISTS key (returns true)		key: "foo" (TTL: 10s)
10s)		
	DELETE key	(key deleted)
	SET key="bar" EX 30	key: "bar" (TTL: 30s)
30s)		
SET key="update" KEeptTL (TTL: 30s)		key: "update"
(KEeptTL uses 30s from B!)		

Problem: Client A intended to preserve a 10-second TTL, but got 30 seconds instead because Client B changed the key between EXISTS and SET.

Why Redis Doesn't Solve This Natively:

Redis is single-threaded **per request**, but:

- 1. Multiple clients can send commands
- 2. Each command is atomic **individually**
- 3. But **sequences** of commands are not atomic
- 4. Redis sees each command as independent

Part 4: Production-Ready Solution: Lua Scripting

Why Lua Script Works:

Atomicity Guarantee: When you send a Lua script to Redis, the **entire script executes atomically**:

- Redis blocks all other clients until script completes
- No interleaving possible
- All-or-nothing execution

Why It Solves Our Race Condition: All three operations (check TTL, check existence, set value) happen in ONE atomic transaction.

Implementation:

```
-- save as: redis/setWithConditionalTTL.lua
--
-- Purpose: Set a key with TTL preservation logic
--
-- Args:
--   KEYS[1] = the key name
--   ARGV[1] = the new value
--   ARGV[2] = default TTL in seconds (30)
--
-- Returns:
--   1 = key was created with default TTL
--   2 = key existed, TTL was preserved
--   0 = error
--
-- Internal Redis Data Structures (simplified):
--   - Each key has: [value, expiry_time_ms, other_metadata]
--   - expiry_time_ms = 0 means no expiration

-- Check if key exists
local existing_ttl = redis.call('PTTL', KEYS[1])
-- PTTL returns:
--   -2 = key doesn't exist
--   -1 = key exists with no TTL
--   n  = milliseconds until expiry

-- Determine action based on existing TTL
if existing_ttl == -2 then
  -- Key doesn't exist: create with default TTL
  redis.call('SET', KEYS[1], ARGV[1], 'EX', ARGV[2])
  return 1
elseif existing_ttl == -1 then
  -- Key exists but has no TTL: preserve (still no TTL)
  redis.call('SET', KEYS[1], ARGV[1], 'KEEPTTL')
  return 2
else
  -- Key exists with TTL: preserve it
  redis.call('SET', KEYS[1], ARGV[1], 'KEEPTTL')
  return 2
end
```

Node.js Usage (Latest node-redis v4+):

```
import { createClient } from "redis";
import { readFileSync } from "fs";
import { join } from "path";
```

```
const redis = createClient({ url: "redis://localhost:6380" });
await redis.connect();

// Load the Lua script
const luaScript = readFileSync(
  join(import.meta.dirname, "redis", "setWithConditionalTTL.lua"),
  "utf-8"
);

// Register script (Redis caches it by SHA1)
const scriptSHA = await redis.scriptLoad(luaScript);

/**
 * Atomic function: Set value, conditionally preserve or create TTL
 *
 * @param {string} key - Redis key name
 * @param {string} value - New value to set
 * @param {number} defaultTTL - TTL in seconds if key doesn't exist
 * (default: 30)
 * @returns {Promise<number>} 1 if created, 2 if updated with TTL
 * preserved
 */
async function setWithConditionalTTL(key, value, defaultTTL = 30) {
  try {
    // Execute script atomically
    // redis.evalSha(SHA, { keys, arguments })
    const result = await redis.evalSha(scriptSHA, {
      keys: [key], // KEYS[1]
      arguments: [value, String(defaultTTL)], // ARGV[1], ARGV[2]
    });

    return result;
  } catch (error) {
    // If script not cached, fallback to loading it
    if (error.message.includes("NOSCRIPT")) {
      return await redis.eval(luaScript, {
        keys: [key],
        arguments: [value, String(defaultTTL)],
      });
    }
    throw error;
  }
}

// Usage Example
await setWithConditionalTTL("rate:user:123", "5", 30);
// Returns: 1 (created with 30s TTL)

// Access again within TTL window
await setWithConditionalTTL("rate:user:123", "6", 30);
// Returns: 2 (value updated, TTL preserved)
```

Part 5: Redis Internals — What Happens Under the Hood

Scenario 1: Key Doesn't Exist

Command: SET mykey "hello" EX 30

Redis Internal State Before:
(key doesn't exist in hash table)

Redis Internal State After:
Hash Table Entry:

```
{
  key: "mykey",
  value: "hello",
  expiry_at_ms: (now_ms + 30000),
  encoding: "raw",
  type: "string",
}
```

Expiration Background Job:
Redis has a background thread that:

- Every 100ms, samples random keys
- Checks if expiry_at_ms <= current_time_ms
- If expired, removes entry
- Also runs active expiration on access

Scenario 2: Key Exists WITHOUT TTL

Before: SET mykey "value1" (no expiry option used)

Redis Internal State:

```
{
  key: "mykey",
  value: "value1",
  expiry_at_ms: 0, // 0 = no expiration
  type: "string",
}
```

Command: SET mykey "value2" KEeptTL

Redis Internal State After:

```
{
  key: "mykey",
  value: "value2",
  expiry_at_ms: 0, // Still no expiration (preserved)
  type: "string",
}
```

Scenario 3: Key Exists WITH TTL

Before: SET mykey "value1" EX 60
(Created at time T, will expire at T+60)

Redis Internal State at T+30:

```
{
  key: "mykey",
  value: "value1",
  expiry_at_ms: T+60000,
  type: "string",
}
```

Command (at T+30): SET mykey "value2" KEEPTTL

Redis Internal State After:

```
{
  key: "mykey",
  value: "value2",
  expiry_at_ms: T+60000, // Original expiry time preserved
  type: "string",
}
```

Note: Only 30 seconds remain, NOT 60
This is why KEEPTTL is valuable for rate limiters:

- First request: increments counter at 0:00, TTL = 60s
- Requests at 0:15, 0:30, 0:45: only 45s, 30s, 15s remain
- At 1:00: counter expires

Part 6: Why Lua is Atomic (The Deep Dive)

Redis Execution Model:

Without Lua (separate commands):

Time	Client A	Redis State	Client B
T0		key: "v1" TTL:10s	
T0.1	EXISTS key	returns 1	
			DELETE key
T0.2	SET KEEPTTL	(oops, wrong TTL)	
		key: "v2" TTL:0	

With Lua (atomic script):

Time	Client A	Redis	Client B
-----	-----	-----	-----

T0		key: "v1" TTL:10s	
T0.1	EVALSHA script	[LOCKED]	queued...
T0.15		PTTL -> 9500ms	
		SET KEPTTL	still locked
T0.2	<- returns 2	[UNLOCKED]	now executed
		key: "v2" TTL:9.5s (preserved!)	

How Redis Ensures This:

1. **Single-threaded core:** Redis has one thread handling commands
2. **Script is compiled:** Lua script is sent once, then referenced by SHA
3. **Blocking guarantee:** While script runs, other commands wait
4. **Atomicity guarantee:** Script completes fully before next command starts

This is why Lua script is production-safe:

- No timing windows between checks
- No interleaving possible
- All 3 operations (check, decide, set) happen together

Part 7: Trade-offs Analysis for System Design Interviews

Approach 1: Simple SET EX (Naive)

```
await redis.set(key, value, { EX: 30 });
```

Pros:

- Simple, readable code
- Fast (1 Redis command)
- Perfect if you DON'T need to preserve TTL

Cons:

- Resets TTL every time
- Doesn't solve your use case

When to use: Cache warming, short-lived data, new session tokens

Approach 2: Separate EXISTS + SET (UNSAFE)

```
const exists = await redis.exists(key);
if (exists) {
  await redis.set(key, value, { KEPTTL: true });
} else {
```

```
    await redis.set(key, value, { EX: 30 });  
  }
```

Pros:

- Intuitive logic
- Solves the problem if single-threaded

Cons:

- **Race conditions in multi-client scenario**
- Wrong TTL can leak in
- Fails under load

When to use: Single-threaded applications, testing only (NOT production)

Approach 3: Redis NX + XX (Hybrid)

```
// First request: create  
await redis.set(key, value, { EX: 30, NX: true });  
// Later requests: update  
await redis.set(key, value, { KEeptTL: true, XX: true });
```

Pros:

- Two separate clients can coordinate
- Each operation is atomic

Cons:

- Still requires TWO Redis commands
- Client must know if first or not
- Doesn't work if you don't know state

When to use: Coordinating multiple clients with known state

Approach 4: Lua Script (PRODUCTION-READY)

```
const result = await redis.evalSha(scriptSHA, {  
  keys: [key],  
  arguments: [value, "30"],  
});
```

Pros:

- Truly atomic

- No race conditions
- Single round-trip to Redis
- Deterministic behavior under load
- Perfect for rate limiters

Cons:

- Slightly more code
- Lua syntax to learn
- Minimal performance overhead (negligible)

When to use: Production backends, rate limiters, distributed counters, financial systems

Part 8: Practical Rate Limiter Example

```
/**
 * Production Rate Limiter using Lua
 *
 * Strategy: Token bucket per user
 * - Each key stores: current_tokens
 * - TTL: sliding window (resets on activity)
 * - First request in window: init to max
 * - Subsequent requests: check and decrement while preserving TTL
 */

async function checkRateLimit(userId, maxRequests = 10, windowSeconds =
60) {
  const key = `rate:${userId}`;

  // Lua script: Check + increment atomically
  const script = `
    local key = KEYS[1]
    local maxRequests = tonumber(ARGV[1])
    local windowSeconds = tonumber(ARGV[2])

    local current = redis.call('GET', key)

    if not current then
      -- First request in window
      redis.call('SET', key, '1', 'EX', windowSeconds)
      return {1, maxRequests - 1} -- [used, remaining]
    else
      -- Subsequent request
      local used = tonumber(current)
      if used >= maxRequests then
        -- Rate limit exceeded
        return {0, 0} -- [allowed, remaining]
      else
        -- Increment while preserving TTL
        redis.call('INCR', key)
        redis.call('SET', key, tostring(used + 1), 'KEEPTTL')
      end
    end
  `
```

```
        return {1, maxRequests - (used + 1)} -- [allowed, remaining]
      end
    end
  `;

  const result = await redis.eval(script, {
    keys: [key],
    arguments: [maxRequests.toString(), windowSeconds.toString()],
  });

  const [allowed, remaining] = result;
  return {
    allowed: allowed === 1,
    remaining: Math.max(0, remaining),
    retryAfter: allowed === 1 ? null : windowSeconds,
  };
}

// Usage
const result = await checkRateLimit("user:123", 10, 60);
if (result.allowed) {
  // Process request
  console.log(`Request allowed. ${result.remaining} remaining.`);
} else {
  // Reject with retry info
  console.log(`Rate limited. Retry after ${result.retryAfter}s`);
}
```

Part 9: Interview-Ready Summary

Key Concepts to Know:

- 1. **Atomicity:** Operations that complete without interruption from other clients
- 2. **Race Condition:** Timing gap where state changes between your check and action
- 3. **TTL Preservation:** Keeping existing expiration time when updating value
- 4. **Lua Atomicity:** Redis executes entire script without interleaving

Why Production Backends Choose Lua:

System	Concern	Solution
Rate Limiter	Double-counting requests	Lua atomic increment
Session Manager	Concurrent updates	Lua check-and-set
Distributed Lock	Missed lock releases	Lua atomic release
Counter Service	Lost counts under load	Lua atomic updates

Red Flags in Interviews:

- ❌ "I'll just use multiple SET commands" → Race condition!
 - ❌ "I'll check exists then set" → Window between commands!
 - ❌ "I'll use KEEPTTL without checking" → Wrong behavior!
 - ✅ "I'll use a Lua script for atomicity" → Production-ready!
-

Part 10: Testing Your Understanding

Question 1: Why does `redis.exists() + redis.set()` fail in a rate limiter? **Answer:** Another client can delete/modify the key between EXISTS and SET, making your TTL decision stale.

Question 2: How does Lua prevent this? **Answer:** Redis locks out all other clients while executing the script atomically.

Question 3: Why use KEEPTTL instead of just SET EX 30 every time? **Answer:** In rate limiting, you want a sliding window. If a user makes requests at 0s, 10s, 20s with 30s window, the final expiry should be 50s, not 30s from their last request. KEEPTTL preserves this.

Question 4: What's the performance cost of Lua? **Answer:** Minimal. The script is cached by SHA, so it's just one Redis command (like any other). No network overhead.

Resources for Further Learning

1. **Redis Documentation:** <https://redis.io/commands/set/>
2. **Redis Lua Scripting:** <https://redis.io/docs/manual/lua-scripting/>
3. **node-redis Lua Docs:** <https://github.com/lonelyplanet/node-redis>
4. **Distributed Systems:** "Designing Data-Intensive Applications" by Martin Kleppmann (Chapter on Atomicity)